Документация к GPU Sandbox

Содержание

1 Назначение программы **2**

2 Архитектура проекта **2**
- 2.1 Память . . . 3
- 2.2 Кэш-память . . . 3
  - 2.2.1 Разбиение адреса и размещение данных . . . 4
  - 2.2.2 Поля и счётчики кэша . . . 5
  - 2.2.3 Поиск строки и политика вытеснения . . . 5
  - 2.2.4 Алгоритм чтения . . . 6
  - 2.2.5 Алгоритм записи . . . 6
  - 2.2.6 Ограничения и сброс . . . 6
- 2.3 Ядро процессора . . . 7
  - 2.3.1 Регистры . . . 7
  - 2.3.2 Программный счётчик . . . 7
  - 2.3.3 Флаги состояния . . . 7
  - 2.3.4 Состояния ядра . . . 7
- 2.4 Конфигурация и подготовка задач . . . 8

3 Инструкции виртуального процессора **8**
- 3.1 Параметры инструкций . . . 9
- 3.2 Поддерживаемые операции . . . 10

4 Симулятор выполнения **10**
- 4.1 Переменные и циклы выполнения . . . 12

5 Проверка корректности входных данных **13**

6 Статистика выполнения **13**

7 Отладочный вывод **14**

8 Сброс и инициализация **14**

9 Тестирование **15**

1 Назначение программы

GPU Sandbox представляет собой программную модель вычислительного ядра, памяти и набора инструкций, реализованную на языке C. Цель проекта — изучение основных принципов работы процессора: адресация памяти, выполнение операций в регистрах, обработка состояний ядра, проверка корректности входных данных и симуляция исполнения программы.

На текущем этапе проект уже содержит не только базовые вычисления, но и элементы условных переходов, проверки ошибок, таймаутов, отладки и автоматического тестирования[1].

2 Архитектура проекта

Модель содержит вычислительные ядра, общую память, программу из набора инструкций и двухуровневый кэш данных.

Таблица 1: Ключевые компоненты архитектуры

Компонент	**Назначение**
`Memory`	Общая виртуальная память с динамически выделяемым массивом 32-битных значений и проверкой границ.
`Cache`	Настраиваемая двухуровневая буферизация: отдельный кэш L1 для каждого ядра и общий кэш L2.
`Core`	Виртуальное вычислительное ядро с регистрами, счётчиком команд, флагом сравнения и состоянием выполнения.
`Instruction`	Структура команды, задающая тип операции и аргументы `a`, `b`, `c`.
`Simulator`	Планировщик задач на нескольких виртуальных ядрах с общей памятью и общим лимитом тактов.
`Statistics`	Набор счётчиков выполненных инструкций, операций и ошибок для каждой задачи.
`SimulatorConfig`	Параметры размера памяти, кэшей, максимального размера программы и лимита тактов; загружаются из JSON.
`Builder`	Вспомогательные функции для создания задач, задания начальных регистров и связывания задач с ядрами.

[1]Подробнее об устройстве ядра, памяти и логике выполнения см. в разделе *Архитектура проекта*.

2.1 Память

`Memory` — это структура виртуальной памяти, содержащая указатель на массив данных `data` и его размер `size`. Память выделяется динамически функцией `memory_init` и освобождается функцией `memory_destroy`.

Память хранит значения типа `uint32_t`.

Операции чтения и записи реализованы следующим образом:

- `memory_write(Memory *memory, uint32_t address, uint32_t value)` — запись 32-битного значения в ячейку памяти;
- `memory_read(const Memory *memory, uint32_t address)` — чтение 32-битного значения из памяти; при недопустимом адресе возвращается ноль;
- `memory_reset(Memory *memory)` — очистка памяти до нуля.

Проверка адреса при выполнении инструкций производится до обращения к памяти. Прямые функции чтения и записи также проверяют границы: чтение возвращает ноль, а запись по недопустимому адресу не выполняется.

2.2 Кэш-память

Кэш хранит копии блоков общей памяти и обслуживает операции `OP_LOAD` и `OP_STORE`. Адрес в этой модели является индексом 32-битного слова, а не байтовым адресом. Параметры каждого уровня задаются конфигурацией: L1 отдельный для каждого ядра, L2 общий для всех ядер.

Таблица 2: Пример организации кэшей при выбранных параметрах

Уровень	**Наборы**	**Пути в наборе**	**Слов в строке**	**Ёмкость данных**
L1, на ядро	32	4	8	1024 слова (4 КиБ)
L2, общий	128	8	16	16 384 слова (64 КиБ)

Здесь *набор* — группа строк, выбранная по части адреса; *путь* — отдельное место для строки внутри набора. Один блок памяти помещается в одну строку. Число путей задаёт, сколько блоков с одинаковым индексом набора могут одновременно находиться в кэше. Для примера возьмём указанные в таблице параметры: ёмкость уровня в словах вычисляется как произведение числа наборов, числа мест в каждом наборе и числа слов в строке:

$$\boxed{\begin{aligned} C_{\text{words}} &= S \cdot E \cdot L, \\ C_{\text{bytes}} &= C_{\text{words}} \cdot 4 \end{aligned}} \tag{1}$$

Здесь C_{words} — ёмкость в словах; C_{bytes} — ёмкость в байтах; S — число наборов; E — число мест (путей) в каждом наборе; L — число слов в одной строке; множитель 4 — число байтов в одном 32-битном слове. Например, при параметрах таблицы для L1 получается $32 \cdot 4 \cdot 8 = 1024$ слова.

2.2.1 Разбиение адреса и размещение данных

Для выбранного уровня адрес слова разбивается на номер блока, индекс набора, метку блока и смещение слова внутри строки. Формулы ниже показывают вычисления, выполняемые функцией `cache_address_decode`:

$$\boxed{\begin{aligned} B &= \left\lfloor \frac{A}{L} \right\rfloor, \quad I = B \bmod S, \\ T &= \left\lfloor \frac{B}{S} \right\rfloor, \quad O = A \bmod L \end{aligned}} \tag{2}$$

Здесь A — адрес слова в памяти; L — число слов в строке выбранного уровня; S — число наборов этого уровня; B — номер блока памяти; I — индекс набора; T — метка, отличающая блоки, попавшие в один набор; O — номер слова внутри строки; $\lfloor x \rfloor$ означает округление вниз до целого.

Обратная формула показывает, как восстановить адрес слова по метке, набору и смещению:

$$\boxed{A = (T \cdot S + I) \cdot L + O} \tag{3}$$

Здесь T — метка блока; S — число наборов; I — индекс набора; L — число слов в строке; O — смещение слова в строке; A — восстановленный адрес слова.

Деление и остаток используются напрямую, поэтому вычисления не требуют, чтобы число наборов или длина строки были степенями двойки. Параметры S и L могут отличаться между уровнями, поэтому один и тот же адрес декодируется отдельно для L1 и L2.

Например, для иллюстрации возьмём адрес слова $A = 100$ и параметры из таблицы:

- в L1 с $L = 8$ и $S = 32$: блок 12, набор 12, тег 0, смещение 4;
- в L2 с $L = 16$ и $S = 128$: блок 6, набор 6, тег 0, смещение 4.

То есть запрошенное слово с адресом 100 находится в позиции 4 строки, начинающейся с адреса 96. Границы строк уровней могут не совпадать; при показанных длинах строк 8 и 16 каждая строка L1 укладывается в одну строку L2.

Строки и слова кэша размещены в двух плоских массивах, а не в массиве структур «набор со встроенными словами». Индекс строки в массиве метаданных вычисляется так:

$$\boxed{R = I \cdot E + W_i} \tag{4}$$

Здесь R — индекс строки в массиве метаданных; I — индекс набора; E — число мест в наборе; W_i — индекс выбранного места, начиная с нуля.

Элемент `Cache.lines[R]` содержит метку блока, признак действительности и время последнего использования. Данные строки лежат отдельно в `Cache.words`; индекс слова в этом массиве равен:

$$\boxed{Q = R \cdot L + O} \tag{5}$$

Здесь Q — индекс слова в массиве данных кэша; R — индекс строки; L — число слов в строке; O — смещение слова внутри строки.

Поэтому все L слов строки занимают непрерывный участок массива, начиная с позиции $R \cdot L$. Для идентификации строки одновременно должны совпасть индекс набора и метка блока, а признак действительности должен быть установлен.

2.2.2 Поля и счётчики кэша

Таблица 3: Основные поля структур кэша

Структура	**Поле**	**Назначение**
`Cache`	`lines`, `words`	Динамические массивы метаданных строк и слов данных.
`Cache`	`set_count`, `way_count`, `line_words`	Геометрия кэша: наборы, пути в наборе и слова в строке.
`Cache`	`access_clock`	Логический счётчик обращений, используемый для определения LRU.
`CacheLine`	`valid`	Показывает, содержит ли строка загруженный блок.
`CacheLine`	`tag`	Идентифицирует блок, размещённый в данном наборе и пути.
`CacheLine`	`last_used`	Значение часов при последнем попадании или заполнении строки.
`CacheAddress`	`set_index`, `tag`, `word_offset`	Результат декодирования адреса для конкретного уровня кэша.
`CacheStatistics`	`hits`, `misses`, `evictions`	Число попаданий, промахов и вытеснений в этом экземпляре кэша.

2.2.3 Поиск строки и политика вытеснения

Функция `cache_probe` просматривает пути только выбранного набора. Если найдена строка с `valid = true` и совпадающим тегом, происходит попадание `CACHE_HIT`. Иначе фиксируется промах: сначала выбирается первый свободный путь с `valid = false`; только если свободных путей нет, выбирается кандидат на вытеснение.

Для выбора строки используется политика «давно не использовалась» (LRU, от анг. *Least Recently Used*). Её идея: при нехватке места удалить тот блок, к которому дольше всего не обращались. Для этого счётчик `access_clock` отдельно для каждого кэша отмечает порядок событий доступа. При попадании и заполнении строки счётчик увеличивается, а новое значение записывается в `last_used` этой строки. При вытеснении выбирается строка набора с наименьшим `last_used`. Свободное место всегда используется раньше, чем удаляется действительный блок.

Счётчики относятся к отдельному объекту `Cache`: у каждого L1 и у общего L2 своя структура статистики. `hits` и `misses` увеличиваются внутри `cache_probe`, а `evictions` — при заполнении пути, который до этого уже был действителен. Следовательно, промах означает неудачный поиск строки, а вытеснение — фактическую замену ранее загруженного блока. Эти счётчики отражают проверки кэша, а не чис-

ло инструкций задачи; например, при промахе L1 запрос слов строки к L2 может вызвать несколько проверок L2.

2.2.4 Алгоритм чтения

Обычная функция `cache_read_word` работает с одним кэшем: проверяет допустимость адреса, декодирует его, ищет строку и при попадании возвращает слово из массива `Cache.words`. При промахе она загружает целую строку из основной памяти и затем возвращает слово по `word_offset`. Этот вариант используется как одноуровневая операция.

В симуляторе чтение выполняется через `cache_read_word_with_l2`:

1. По адресу ищется строка L1. При попадании нужное слово сразу возвращается ядру.
2. При промахе L1 выбирается свободный путь или LRU-путь для будущей строки L1.
3. Для каждого слова строки L1 вызывается чтение одного слова через кэш L2. При попадании L2 слово берётся из L2; при промахе L2 его строка целиком заполняется из основной памяти.
4. Собранные слова копируются в выбранную строку L1, после чего запрошенное слово возвращается ядру.

Например, если длина строки L1 равна 8 словам, а L2 — 16 словам, строка L1 располагается внутри одной строки L2. При промахе L1 первый запрос может привести к загрузке строки L2, а следующие слова обычно будут найдены в уже загруженной строке L2.

2.2.5 Алгоритм записи

Запись использует сквозную политику: каждое новое значение сразу записывается не только в кэш, но и в основную память (от англ. — *write-through*). Благодаря этому основная память не ждёт вытеснения строки, чтобы получить обновлённое значение. Применяется и выделение строки при промахе (*write-allocate*): если нужного блока нет в L1, строка сначала загружается в L1, затем изменяются слово кэша и основная память. Функция `cache_write_word_with_l2` после этого обновляет соответствующее слово L2, если строка там уже есть, или загружает строку L2 из уже обновлённой памяти. При успешном завершении значения в L1 этого ядра, L2 и основной памяти совпадают.

2.2.6 Ограничения и сброс

При заполнении `cache_fill` загружает всю строку и требует, чтобы весь блок помещался в памяти. Поэтому допустимый адрес последнего слова памяти может привести к ошибке кэша, если его неполная крайняя строка выходит за границу массива. Например, при размере памяти 262 144 слова и длинах строк 8 и 16 край памяти совпадает с границей строк, поэтому неполного блока нет.

Кэши L1 принадлежат ядрам, но автоматический протокол когерентности между ними не реализован. Запись одного ядра обновляет его L1, общий L2 и основную

память, однако не обновляет и не инвалидирует копии в L1 других ядер. Если другое ядро уже сохранило эту строку у себя, оно может прочитать устаревшее значение из собственного L1. Прямые вызовы функций памяти также не синхронизируют кэши.

Функция `cache_invalidate` делает недействительной найденную строку только в переданном экземпляре кэша; её содержимое и счётчики не очищаются. Если строка не найдена, операция всё равно считается успешно выполненной. `cache_reset` очищает метаданные, слова и статистику, сбрасывает `access_clock`, но сохраняет параметры геометрии кэша. Основная память при сбросе кэша не изменяется.

2.3 Ядро процессора

`Core` — структура вычислительного ядра. В ней хранятся регистры, программный счётчик, текущее состояние и флаг сравнения.

2.3.1 Регистры

`registers[REGISTERS_COUNT]` — массив регистров, который используется для промежуточных вычислений и передачи данных между инструкциями.

Количество регистров задаётся константой `REGISTERS_COUNT`, равной 8.

2.3.2 Программный счётчик

`pc` (программный счётчик) — счётчик, который хранит индекс текущей выполняемой команды. После выполнения инструкции счётчик изменяется в соответствии с состоянием ядра.

2.3.3 Флаги состояния

Внутри ядра присутствует переменная `flag`, которая используется для результатов сравнения. В проекте применяются следующие значения:

- `SAME` — значения равны;
- `GREATER` — первое значение больше второго;
- `LESS` — первое значение меньше второго.

2.3.4 Состояния ядра

`current_state` управляет поведением выполнения программы. Набор состояний представлен в перечислении `State`:

1. `NORMAL` — обычный режим работы ядра;
2. `JUMP` — выполнение перехода на другую инструкцию;
3. `HALT` — завершение работы программы;
4. `ERROR` — аварийное состояние при ошибке выполнения;
5. `TIMEOUT` — превышен лимит количества циклов.

При возникновении ошибки, например, обращении к недопустимому регистру или адресу памяти, делении на ноль либо неизвестном опкоде, состояние переводится в ERROR. Лимит тактов задаётся параметром конфигурации max_cycles; при его достижении работающие задачи получают статус таймаута.

2.4 Конфигурация и подготовка задач

Параметры симуляции хранятся в JSON-файле src/components/config/config.json и загружаются функцией config_load_file. Версия формата конфигурации должна быть равна 1. В файле задаются размеры и организация L1/L2, размер памяти, максимальный размер программы и максимальное число тактов. Для текущей конфигурации установлены 262 144 слова памяти, 256 инструкций на программу и 1 000 000 тактов. При запуске симулятор проверяет, что размер каждой программы не превышает настроенный предел.

Модуль Builder предоставляет функции для подготовки входных данных: task_create создаёт задачу со статусом FREE либо с явно заданным статусом, task_set_register задаёт начальное значение регистра, simulator_configure связывает симулятор с массивами ядер и задач, а memory_load_words загружает последовательность значений в память с проверкой границ.

3 Инструкции виртуального процессора

Структура инструкции представлена типом Instruction[2]. Максимальный размер программы задаётся параметром max_program_size в конфигурации и проверяется перед запуском симуляции.

```
typedef struct{
    Opcode opcode;
    uint32_t a;
    uint32_t b;
    uint32_t c;
} Instruction;
```

В проекте используются следующие значения опкодов:

1. OP_LOAD
2. OP_STORE
3. OP_ADD
4. OP_SUB
5. OP_MUL
6. OP_DIV
7. OP_HALT
8. OP_JUMP

[2]Подробное описание набора операций и их логики работы представлено в таблице инструкций.

9. `OP_CMP`
10. `OP_JIF`

3.1 Параметры инструкций

В проекте применяется следующий порядок использования операндов:

- `a` — целевой регистр, регистр с адресом памяти или индекс инструкции, в зависимости от операции;
- `b` — регистр с адресом памяти, исходный регистр или ожидаемое значение флага для `OP_JIF`;
- `c` — второй источник данных при арифметических операциях.

3.2 Поддерживаемые операции

Таблица 4: Набор инструкций

Операция	a	b	c	Назначение
OP_LOAD	регистр назначения	регистр с адресом	—	Чтение значения из memory[reg[b]] в reg[a]
OP_STORE	регистр с адресом	регистр со значением	—	Запись значения reg[b] в ячейку memory[reg[a]]
OP_ADD	регистр назначения	регистр 1	регистр 2	Сложение reg[b] + reg[c] и сохранение в reg[a]
OP_SUB	регистр назначения	регистр 1	регистр 2	Вычитание reg[c] из reg[b] и сохранение в reg[a]
OP_MUL	регистр назначения	регистр 1	регистр 2	Умножение reg[b] * reg[c] и сохранение в reg[a]
OP_DIV	регистр назначения	регистр 1	регистр 2	Деление reg[b] / reg[c] с проверкой на деление на ноль
OP_JUMP	индекс инструкции	—	—	Безусловный переход к инструкции с индексом a
OP_CMP	регистр 1	регистр 2	—	Сравнение значений регистров и установка флага flag
OP_JIF	индекс инструкции	значение флага	—	Условный переход: если (Flag)b == core->flag, ядро переходит к инструкции с индексом a
OP_HALT	—	—	—	Завершение выполнения программы и перевод ядра в состояние HALT

Для OP_JIF поле b содержит значение перечисления Flag, а не номер регистра. При совпадении флага учитывается условный переход, даже если его цель совпадает с текущей инструкцией. Некорректные индексы регистров, адреса памяти и цели переходов переводят ядро в состояние ERROR.

4 Симулятор выполнения

Модуль simulator выполняет задачи на нескольких виртуальных ядрах. Программа представлена структурой Program (указатель на инструкции и их количество),

а структура `Task` хранит программу, начальные значения регистров, статус и статистику задачи. Структура `CoreContext` связывает виртуальное ядро с назначенной задачей и её кэшем L1. Структура `Simulator` содержит массивы ядер и задач, общую память, общий кэш L2, индекс следующей задачи и глобальное число тактов. Параметры памяти и кэшей передаются в `run` через структуру `SimulatorConfig`.

Планировщик назначает задачи свободным ядрам в порядке их расположения в массиве задач. За один глобальный такт каждое занятое ядро исполняет не более одной инструкции. Операции выполняются последовательно в порядке обхода массива ядер. Память и кэш L2 общие для всех задач, кэш L1 закреплён за ядром. Начальные регистры задачи копируются в ядро после его сброса.

Для отдельной задачи возможны статусы `FREE`, `WORKING`, `DONE`, `TASK_ERROR` и `TASK_TIMEOUT`. Выполнение задачи заканчивается при достижении конца программы, инструкции `HALT` или ошибке. Общая симуляция завершается, когда обработаны все задачи либо достигнут настроенный лимит `max_cycles`. При достижении лимита задачи, уже выполняющиеся на ядрах, получают статус `TASK_TIMEOUT`; ещё не назначенные ядрам задачи остаются в исходном статусе.

Модуль запускает симуляцию до тех пор, пока не будет выполнен один из следующих сценариев:[3]

- достигнут конец программы;
- ядро перешло в состояние `HALT`;
- возникла ошибка;
- достигнут настроенный лимит `max_cycles`.

Поле `Simulator.cycles` хранит количество глобальных тактов всей симуляции. В текущем файле конфигурации лимит равен 1 000 000 тактов. Дополнительно каждая задача ведёт собственные счётчики вызовов исполнения инструкций и тактов в структуре статистики.

Схема выполнения выглядит так:

1. планировщик назначает свободным ядрам очередные задачи;
2. каждое занятое ядро выполняет одну инструкцию через `step_core` и `execute`;
3. инструкции чтения и записи обращаются к L1, затем при необходимости к общему L2 и основной памяти;
4. после инструкции обновляются счётчики статистики, программный счётчик и статус задачи;
5. завершившиеся или ошибочные задачи освобождают ядра;
6. после обхода ядер увеличивается общий счётчик тактов `Simulator.cycles`;
7. при достижении общего лимита работающие на ядрах задачи получают статус `TASK_TIMEOUT`.

[3]Более подробно тесты многозадачной симуляции описаны в разделе *Тестирование*.

4.1 Переменные и циклы выполнения

Циклы в симуляторе нужны для повторения одного и того же шага над меняющимся набором задач: внешний цикл двигает общее время вперёд, внутренний обходит ядра, а планировщик назначает освободившимся ядрам следующие задачи. Без этих циклов пришлось бы заранее знать число задач и вручную вызывать исполнение каждой инструкции.

Таблица 5: Переменные управления симуляцией и учёта времени

Переменная	**Где хранится**	**Для чего нужна**
`pc`	`Core`	Индекс следующей инструкции текущей программы; меняется при обычном ходе или переходе.
`next_task`	`Simulator`	Индекс задачи, которая ещё не назначена ядру; сдвигается после назначения задачи.
`task_index`	`CoreContext`	Связывает занятое ядро с элементом массива задач, которой оно сейчас исполняет.
`Simulator.cycles`	`Simulator`	Число уже завершённых глобальных шагов симуляции; сравнивается с настроенным `max_cycles`.
`Statistics.cycles`	Каждая задача	Число инструкций, переданных на исполнение этой задачи; увеличивается вместе с `instruction_executed`.
`Cache.access_clock`	Каждый объект `Cache`	Логический номер события доступа к кэшу для выбора наименее недавно использованной строки.

Внешний цикл `while` в `run` продолжается, пока есть работающие ядра или неназначенные задачи и одновременно не исчерпан лимит тактов. В начале каждого шага функция `submin_task` ищет свободные ядра и выдаёт им задачи по порядку. Затем цикл `for` проходит по массиву ядер: занятое ядро делает не более одного вызова `step_core`, поэтому все активные ядра продвигают свои программы на одну инструкцию за глобальный шаг. Завершившаяся или ошибочная задача освобождает ядро; на следующем шаге это ядро может получить следующую задачу.

Условие внешнего цикла можно представить формулой:

$$\boxed{\begin{array}{l}(\text{есть работающие ядра} \ \vee \ \text{есть неназначенные задачи}) \\ \wedge \ (\text{Simulator.cycles} < \text{max_cycles})\end{array}} \tag{6}$$

Здесь «есть работающие ядра» означает, что хотя бы одна задача выполняется; «есть неназначенные задачи» означает, что остались задачи для запуска; `Simulator.cycles` — число завершённых общих тактов; `max_cycles` — заданный в конфигурации предел.

После обхода ядер `Simulator.cycles` увеличивается на единицу. Если задача дошла до конца программы, выполнила `OP_HALT` или получила ошибку, её ядро перестаёт исполнять её. Команды перехода меняют `pc`: при `OP_JUMP` переход безусловный, а при `OP_JIF` он выполняется только при совпадении ожидаемого флага с текущим. Поэтому `while` позволяет выполнять циклы самой виртуальной программы: её инструкции могут возвращать `pc` к более раннему индексу.

Важно не смешивать три счётчика. `Simulator.cycles` считает шаги всей симуляции, в каждом из которых может исполниться по одной инструкции на нескольких ядрах. `Statistics.cycles` конкретной задачи увеличивается на один при каждом вызове исполнения её инструкции; следовательно, это счётчик инструкционных шагов задачи, а не время ожидания задачи между глобальными шагами.

`Cache.access_clock` не является временем процессора: это локальная шкала обращений к одному экземпляру кэша, нужная только для LRU-вытеснения.

5 Проверка корректности входных данных

Для контроля ошибок проект использует набор валидаторов, реализованных в модуле инструкций[4]. Основные проверки:

- `checker_destination` — проверка допустимого индекса целевого регистра;
- `checker_address_register` — проверка адресного регистра;
- `checker_source_register` — проверка регистра-источника;
- `checker_memory_address` — проверка корректности адреса памяти;
- `checker_instruction_index` — проверка целевого индекса перехода в пределах программы.

При ошибке проверки ядро переводится в состояние `ERROR`, а в консоль выводится сообщение о причине сбоя. Ошибкой также завершается неудачное обращение к кэшу при выполнении операции чтения или записи.

6 Статистика выполнения

Для каждой задачи симулятор хранит структуру `Statistics`. Счётчики операций увеличиваются при успешном выполнении соответствующей инструкции. Счётчик ошибок увеличивается при ошибке валидации, делении на ноль, неизвестном опкоде или неудачном обращении к кэшу. Счётчики `instruction_executed` и `cycles` увеличиваются после вызова исполнения инструкции, в том числе если этот вызов завершился ошибкой. Счётчик условных переходов увеличивается при совпадении флага с ожидаемым значением.

[4]Подробнее о логике ошибок и проверках см. раздел *Симулятор выполнения*.

Таблица 6: Счётчики структуры `Statistics`

Поле	Тип	Назначение
`cycles`	`uint32_t`	Число тактов, затраченных на выполнение задачи.
`instruction_executed`	`uint32_t`	Общее число исполненных инструкций задачи.
`loads`, `stores`	`uint32_t`	Число успешно выполненных операций чтения и записи памяти.
`adds`, `subs`	`uint32_t`	Число успешно выполненных сложений и вычитаний.
`muls`, `divs`	`uint32_t`	Число успешно выполненных умножений и делений.
`jumps`	`uint32_t`	Число выполненных безусловных переходов.
`conditional_jumps`	`uint32_t`	Число условных переходов, для которых совпал флаг и условие перехода выполнилось.
`checks`	`uint32_t`	Число успешно выполненных сравнений инструкцией `OP_CMP`.
`errors`	`uint32_t`	Число зарегистрированных ошибок при выполнении инструкций и проверке операндов.

7 Отладочный вывод

Проект содержит набор функций для вывода состояния виртуального процессора и симуляции:[5]

- `print_registers` и `print_flag` — печать регистров и флага ядра;
- `print_state` — печать текущего состояния ядра;
- `print_program` — отображение программы с операциями и аргументами;
- `print_statistics` — вывод статистики отдельной задачи;
- `print_simulator` — вывод общего числа тактов, состояний ядер и статусов задач с их статистикой;
- `print_cache_state`, `print_cache_stats` и `print_memory_state` — вывод содержимого и статистики кэша, а также выбранного диапазона памяти.

8 Сброс и инициализация

Для повторного использования модели предусмотрены функции сброса:

[5] Функции отладки напрямую связаны с режимами работы ядра, описанными в разделе *Архитектура проекта*.

- `core_reset(Core *core)` сбрасывает регистры в ноль, обнуляет счётчик `pc`, устанавливает флаг `SAME` и возвращает состояние ядра в `NORMAL`;
- `memory_reset(Memory *memory)` очищает память;
- `cache_reset(Cache *cache)` очищает строки и счётчики кэша, сохраняя число наборов. При запуске симуляции кэши L1 и L2 заново инициализируются функцией `cache_init`.

Это позволяет запускать несколько тестовых сценариев последовательно на одном и том же экземпляре модели.

9 Тестирование

В папке `tests` находятся отдельные программы, проверяющие корректность ключевых компонентов, конфигурации, подготовки задач и многозадачной симуляции:[6]

- `test_core.c` — проверяет сброс ядра;
- `test_memory.c` — проверяет запись, чтение и очистку памяти;
- `test_cache.c` — проверяет декодирование адресов, попадания и промахи, выбор пути, вытеснение, заполнение строк, запись, инвалидацию и чтение через L2;
- `test_instruction.c` — проверяет основные инструкции, флаги сравнения и обработку ошибок;
- `test_debug.c` — проверяет функции вывода состояния и симулятора;
- `test_simulator.c` — проверяет распределение нескольких задач между ядрами, результаты в общей памяти, обработку ошибочной задачи и таймаут;
- `test_builder.c` — проверяет создание задач, установку регистров, настройку симулятора и загрузку слов в память;
- `test_config.c` — проверяет чтение и разбор конфигурации, а также отклонение неверных параметров.

Автотесты написаны через макросы `assert`, что позволяет проверять корректность поведения программы на уровне условий и побочных эффектов.

[6] См. также раздел *Проверка корректности входных данных*.